DATA 260: Agentic AI

Instructor: Prof. Dr. Simon Shim

Student Name: Trapti Kulshrestha

STUDENT ID: 019124833

Homework: 1

Section 0: Personal Configuration:

SID4	4833
PORT_BASE:	8333
PREFIX	s4833
SEED	4833
VERIFY_SEED	264833
DOMAIN_ID:	1
Assigned Domain	Clinical Trial Listings

Hardware: Apple MacBook Air M4

Local Model: qwen3:8b

Tagged Commit Hash: a1d8ab8f028eeacbdf7b81e8373f5df710006a1c

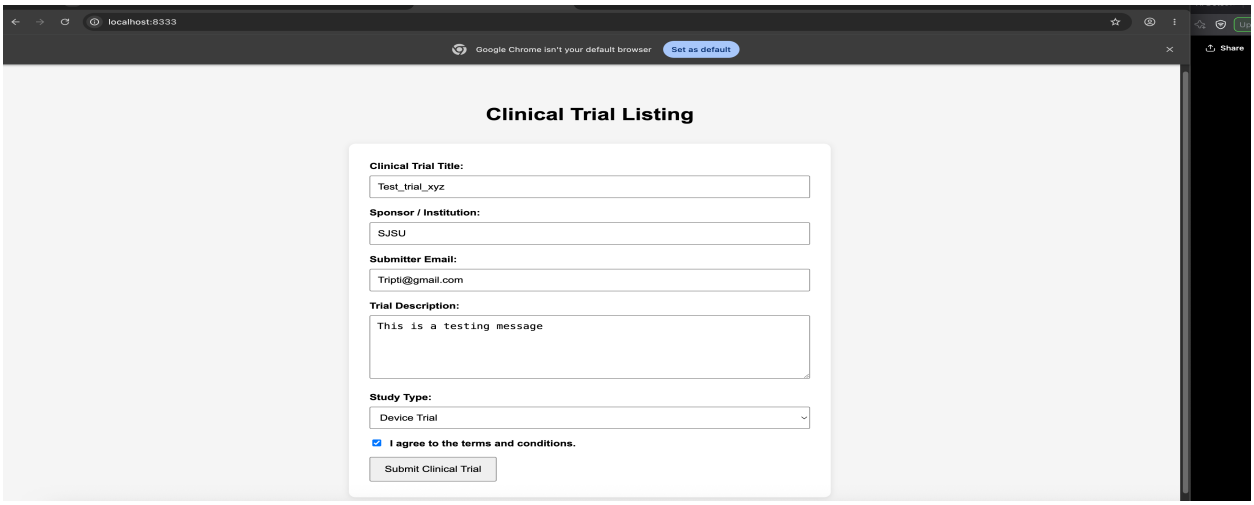
Tagged GitHub Repository: https://github.com/Psycoder0611/data260-4833/tree/hw1

Part 1 – HTML Form and JavaScript

My assigned domain is - *Clinical Trial Listings*. I created a simple web form where a user can enter the trial title, sponsor or institution name, submitter email, trial description, study type and accept the terms and conditions. I used the fields listed in DOMAIN_SCHEMA.md while designing the form.

Field	What I used
Primary text field	Clinical Trial Title
Secondary text field	Sponsor / Institution
Email field	Submitter Email
Textarea	Trial Description
Select menu	Study Type
Checkbox	Terms and Condition

Figure 1. Final Clinical Trial Listing form running locally



I tested form with different inputs to make sure the validation worked correctly. The email field uses the browser's built-in email validation. I also added custom JavaScript checks for the trial description length and the terms-and-conditions checkbox.

Figure 2. Email validation for an invalid email address

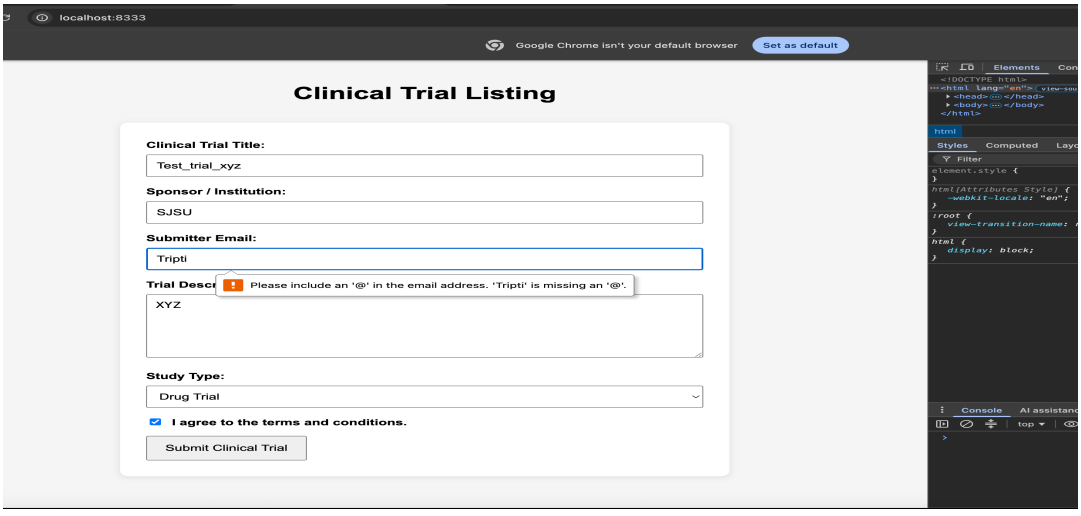


Figure 3. Custom alert when the trial description does not meet the required length

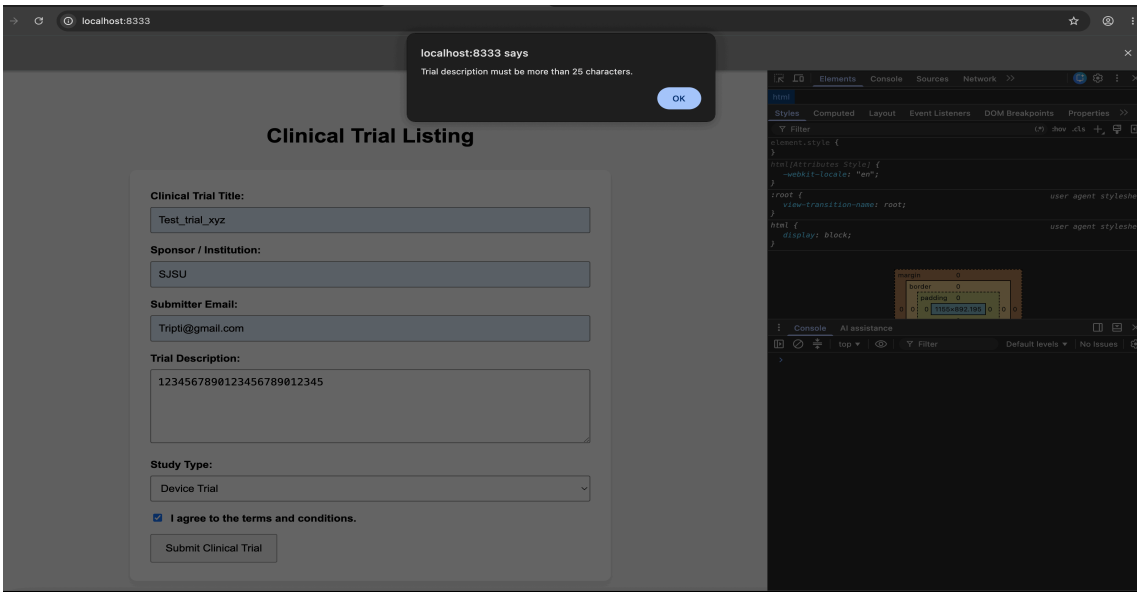
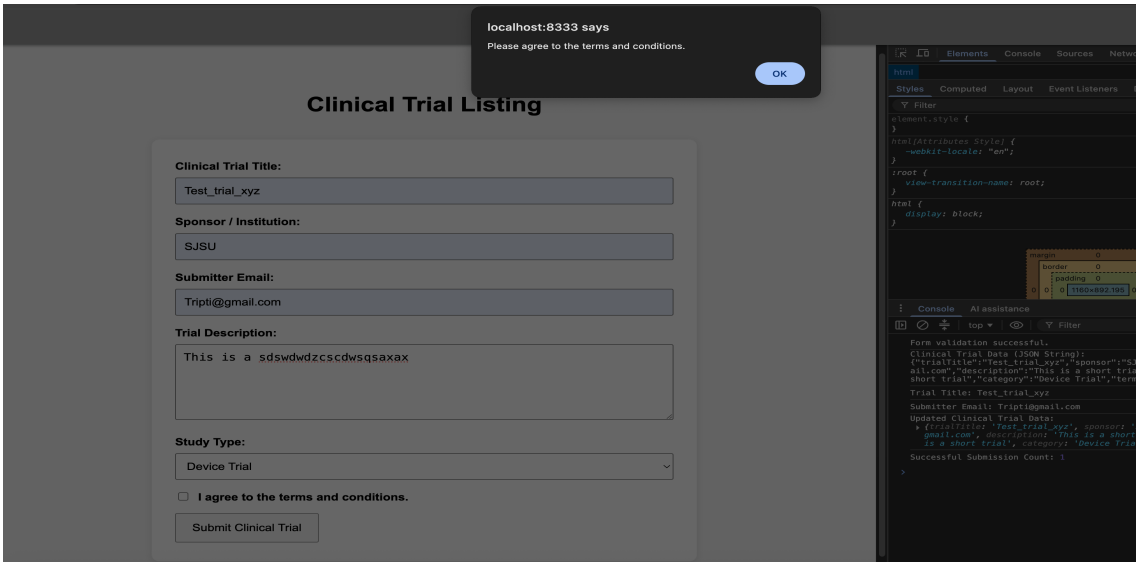
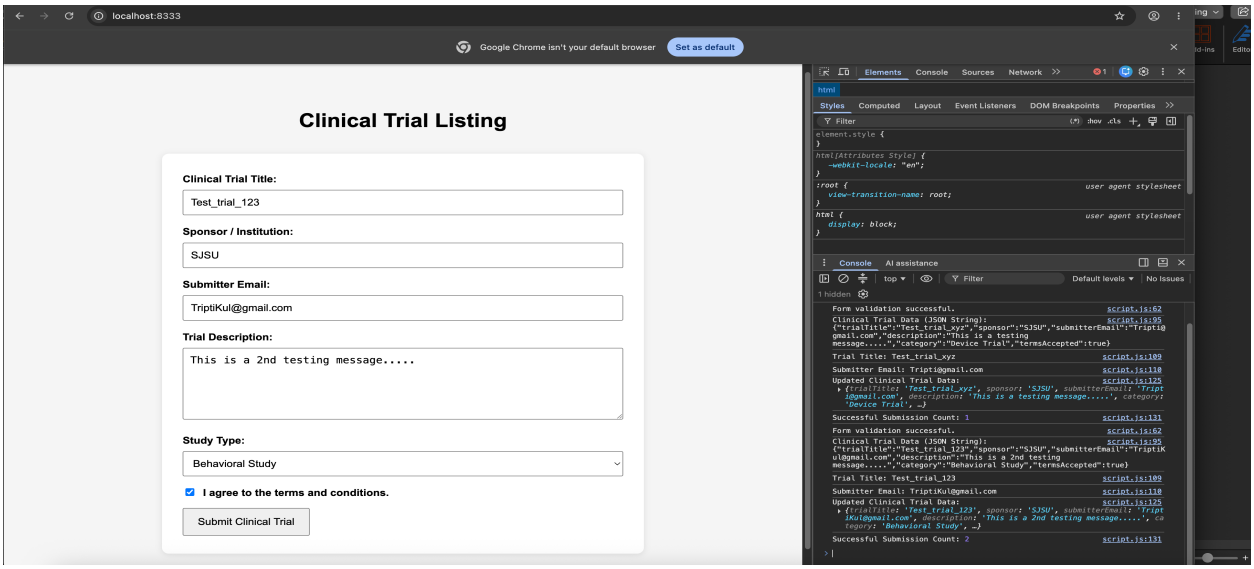


Figure 4. Custom alert when the terms and conditions are not accepted.



After the form passes validation, I collect the entered values into a JavaScript object & convert it to JSON file using `JSON.stringify()`. I then parse the JSON back into an object using `JSON.parse()`. I used object destructuring to extract the trial title & submitter email & the spread operator to add the submission date. I also used a closure to keep track of the number of successful submissions.

Figure 5. Successful form submission showing JSON output, destructuring, updated data, and submission count in the browser console.



Deployment: Part -1: Docker Deployment

After finishing HTML & JavaScript part, I created a Docker image for the application using Nginx. I built the image locally & ran the container by mapping `port 8333` on my machine to `port 80` inside the container.

Figure 6. Clinical Trial Listing form running in the browser on `localhost:8333`

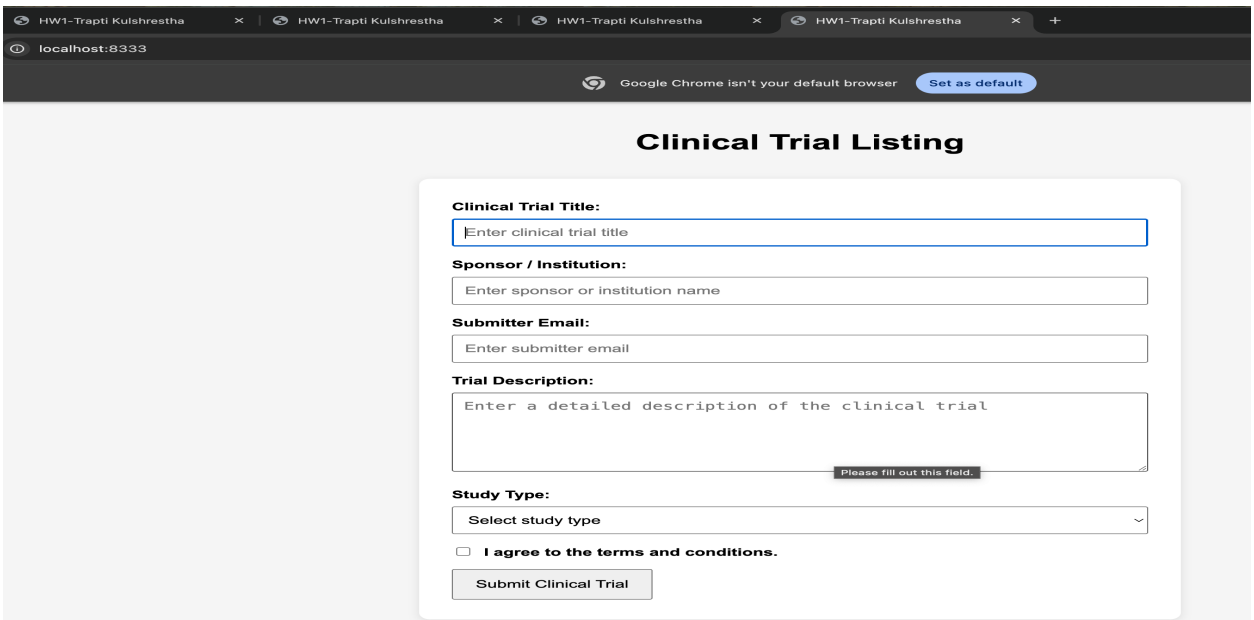


Figure 7: Docker build and container run output for the Clinical Trial Listing application

AWS ECS Deployment

After testing the Docker container locally, I pushed the Docker image to Amazon ECR & deployed it using Amazon ECS with Fargate. I created an ECS cluster, task definition & service & enabled a public IP so the application could be opened from a browser.

Figure 8. Docker image pushed successfully to Amazon ECR

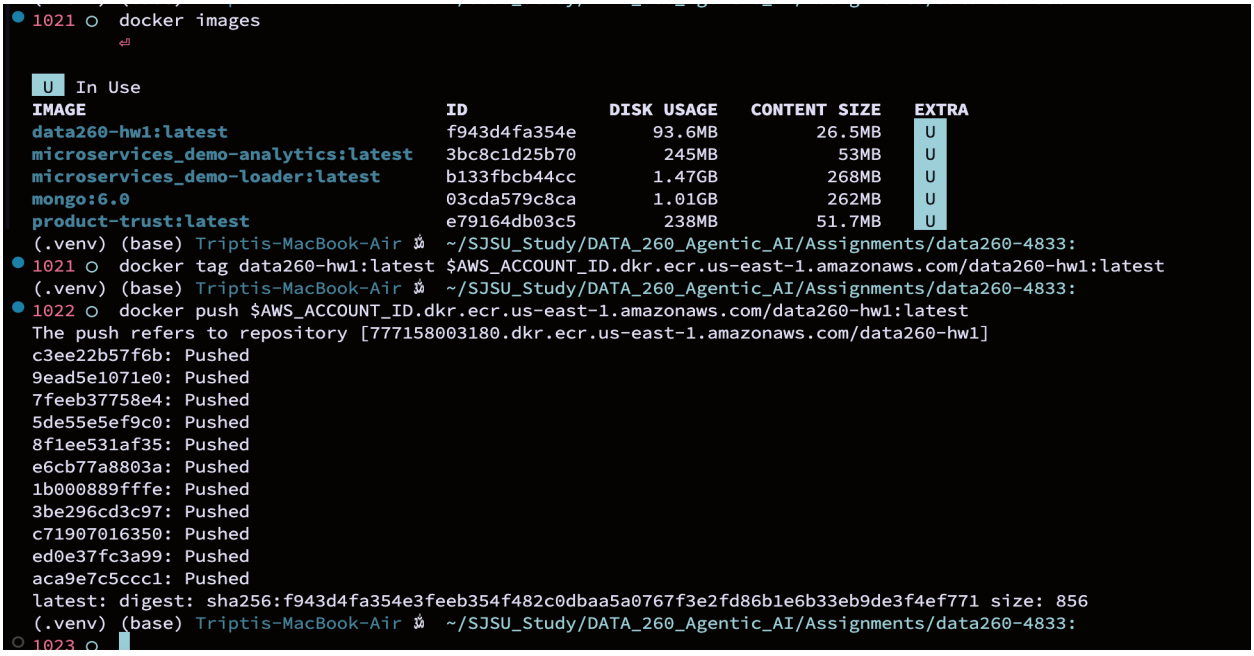


Figure 9. ECS task definition created for the Clinical Trial Listing application

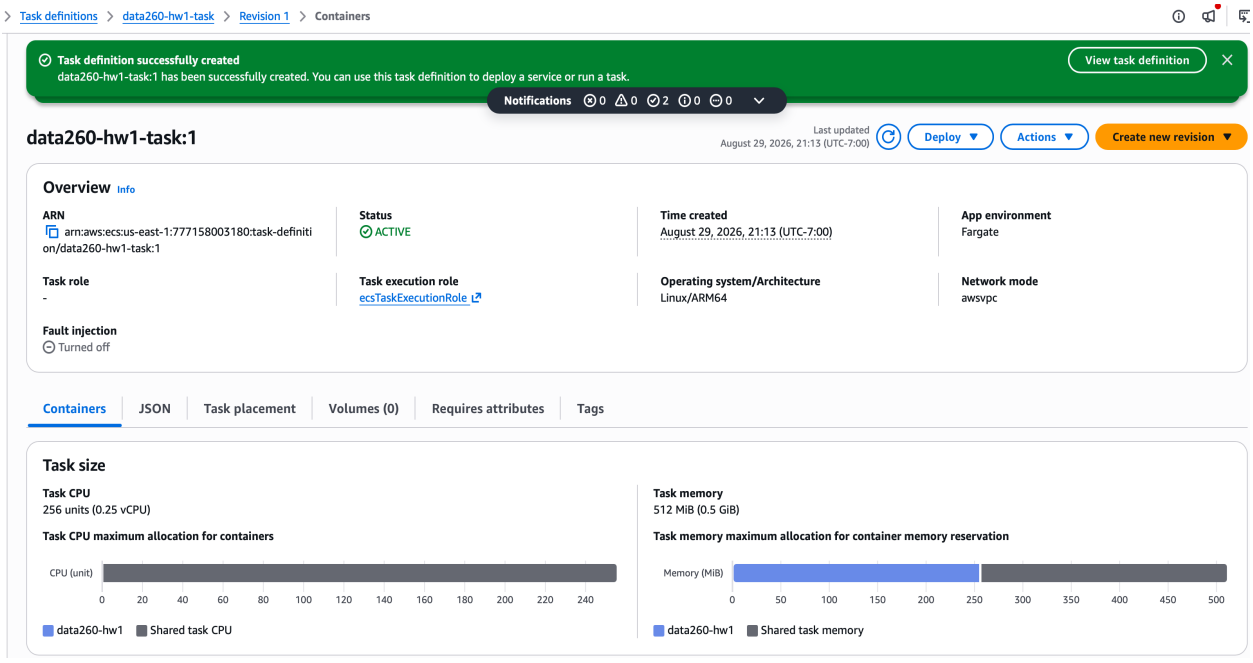


Figure 10. ECS service deployed successfully with one running Fargate task

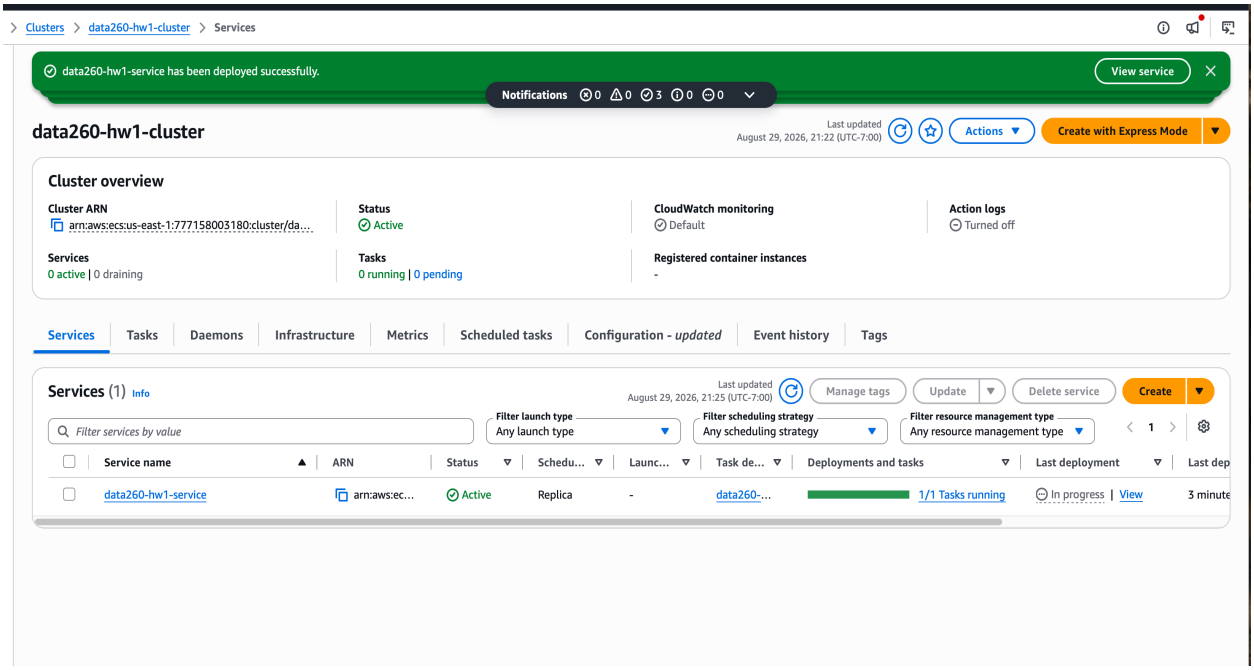


Figure 11. Running ECS task showing the assigned public IP

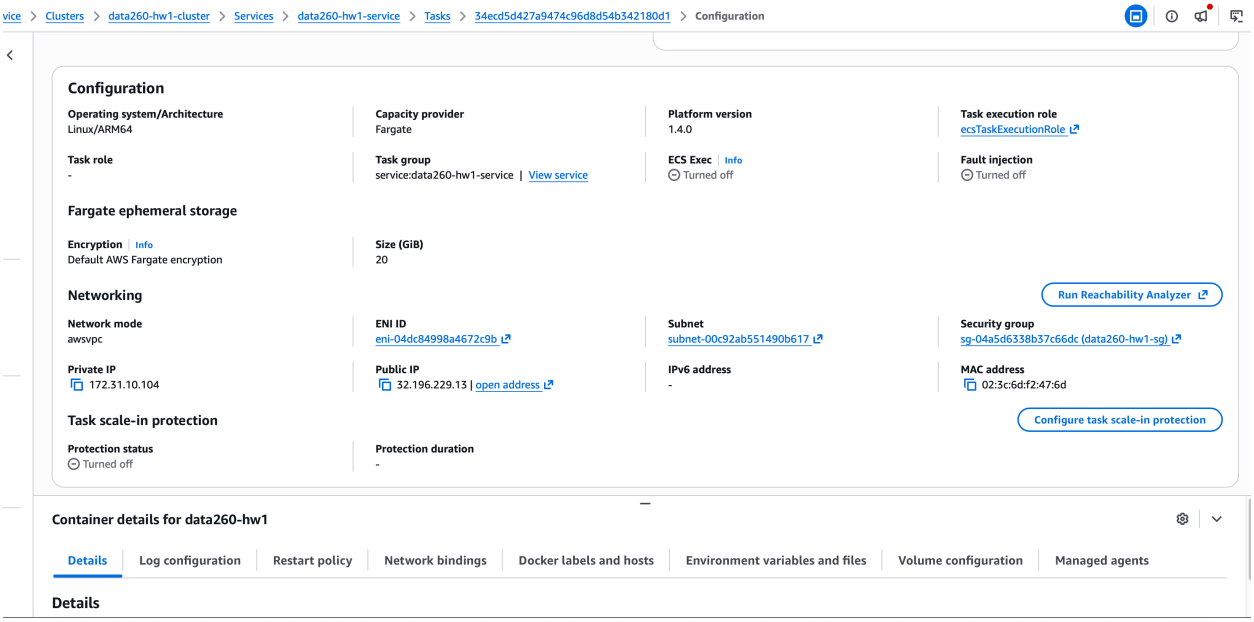
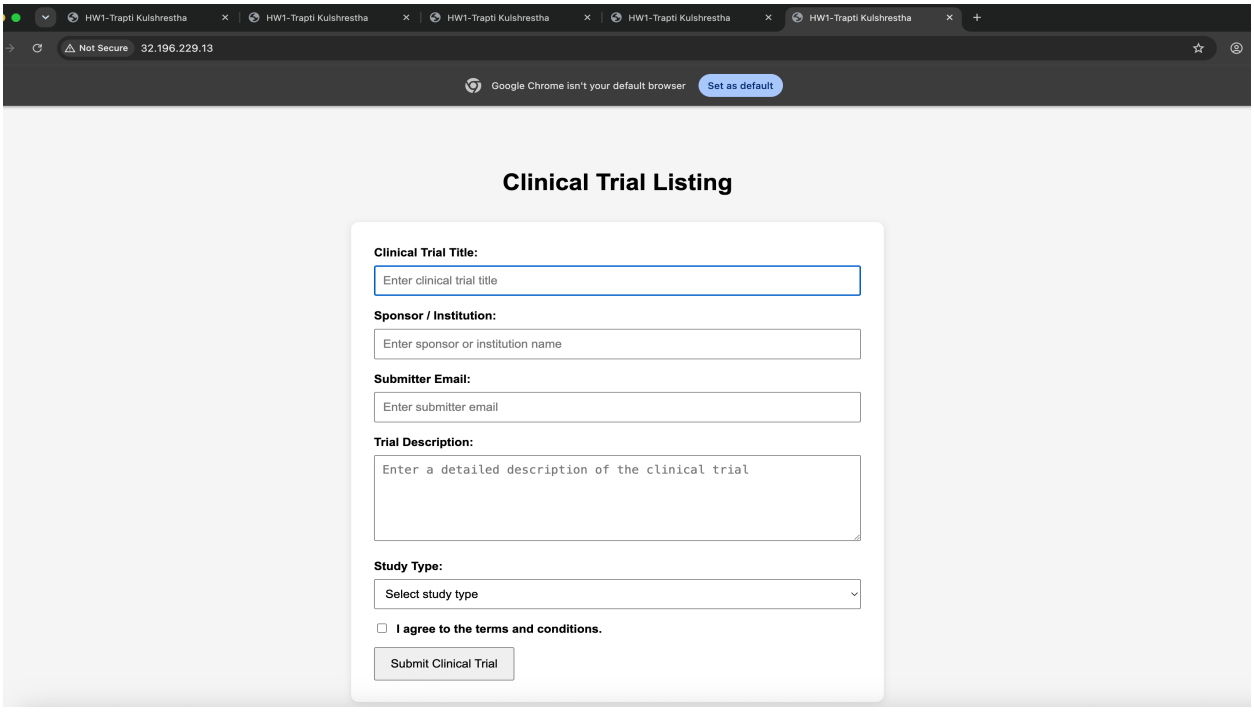


Figure 12. Clinical Trial Listing application running successfully from the ECS public IP

Public IP: http://32.196.229.13



After confirming that the deployment worked, I changed the desired task count to 0 so that the Fargate task would stop running.

PART-2 Agentic AI Workflow

In this part, I used a local Ollama model with a simple multi-agent workflow. The workflow had three agents: Planner, Reviewer, and Finalizer. The Planner generated the first set of tags and summary, the Reviewer checked and improved the result, and the Finalizer produced the final output.

I used qwen3 **:** 8b as the local model. I tested the workflow with the same clinical trial input and checked how the output changed between the Planner, Reviewer and Finalizer stages.

The Reviewer did make changes to the Planner output. It changed both the tags and the summary before the result was passed to the Finalizer. This showed that the agents were not simply repeating the same response, but were reviewing and refining the previous output.

Figure 13. qwen3:8b model available locally through Ollama

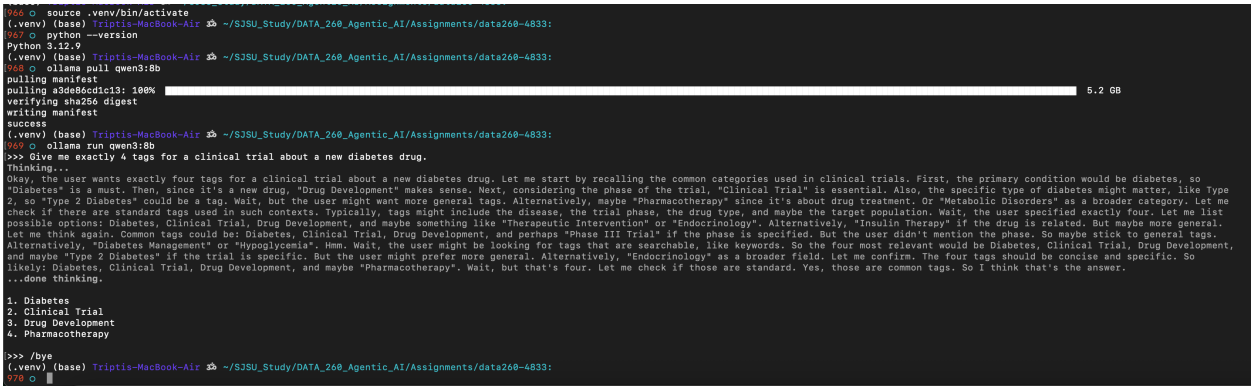


Figure 14. Multi-agent workflow implemented using LangChain and Ollama

Command Used: python agents_demo.py --title "New Diabetes Drug Trial" --content "A clinical trial is evaluating a new diabetes medication designed to improve blood glucose control in adults with type 2 diabetes." --email "tripti@example.com" --model qwen3:8b --strict

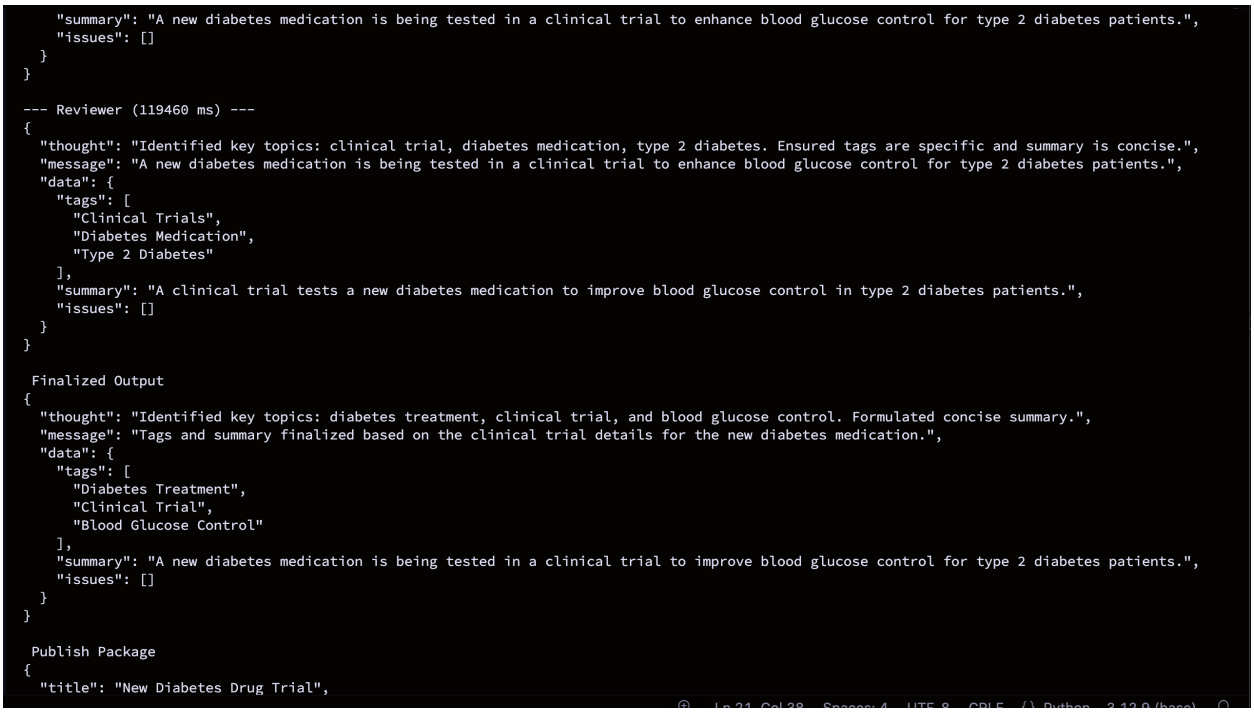
Figure 15. Planner, Reviewer, Finalizer, and final Publish Package output from the multi-agent workflow

```
KeyboardInterrupt

(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:
980 ○ ollama list
NAME        ID              SIZE      MODIFIED
qwen3:8b    500a1f067a9f    5.2 GB    14 hours ago
(.venv) (base) Triptis-MacBook-Air ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:
981 ○ python agents_demo.py --title "New Diabetes Drug Trial" --content "A clinical trial is evaluating a new diabetes medication designed to impro
d glucose control in adults with type 2 diabetes." --email "tripti@example.com" --model qwen3:8b --strict

--- Planner (252471 ms) ---
{
  "thought": "Identified key topics: clinical trial, diabetes medication, and blood glucose control. Formulated relevant multi-word tags and summari
 trial's purpose.",
  "message": "A new diabetes medication is being tested in a clinical trial to enhance blood glucose control for type 2 diabetes patients.",
  "data": {
    "tags": [
      "Clinical Trial Diabetes Drug",
      "Type 2 Diabetes Treatment",
      "Blood Glucose Control Study"
    ],
    "summary": "A new diabetes medication is being tested in a clinical trial to enhance blood glucose control for type 2 diabetes patients.",
    "issues": []
  }
}

--- Reviewer (119460 ms) ---
{
  "thought": "Identified key topics: clinical trial, diabetes medication, type 2 diabetes. Ensured tags are specific and summary is concise.",
  "message": "A new diabetes medication is being tested in a clinical trial to enhance blood glucose control for type 2 diabetes patients.",
  "data": {
    "tags": [
      "Clinical Trials",
      "Diabetes Medication",
      "Type 2 Diabetes"
    ],
    "summary": "A clinical trial tests a new diabetes medication to improve blood glucose control in type 2 diabetes patients.",
```



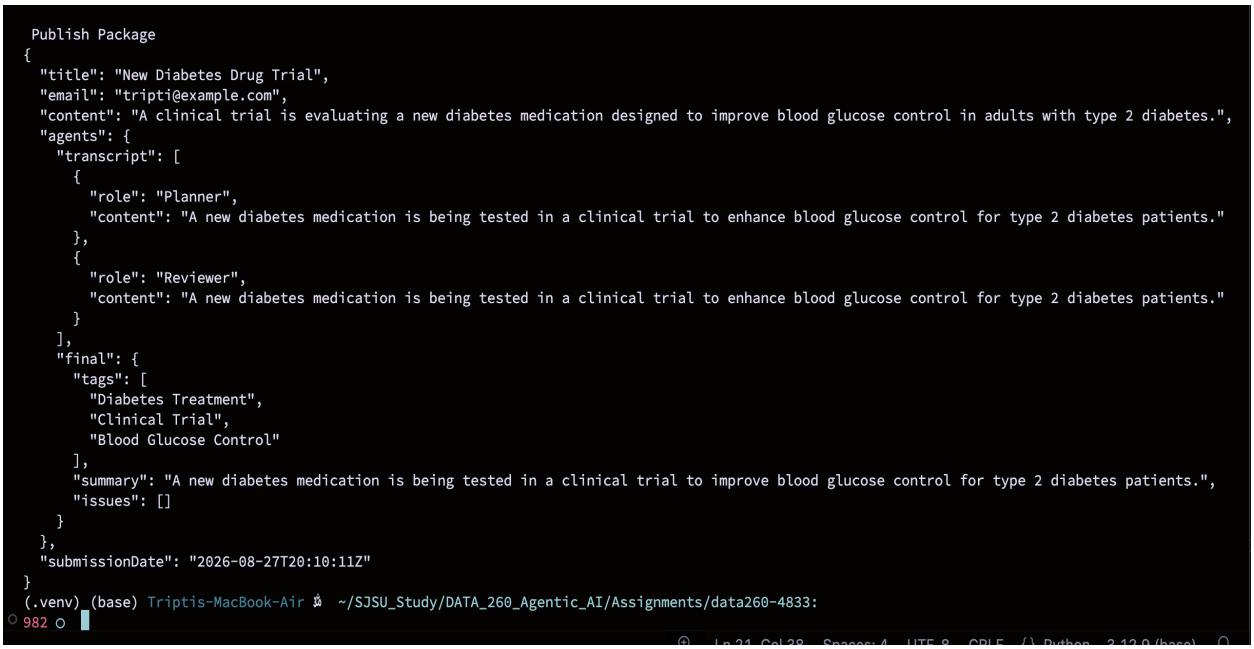
```
    "summary": "A new diabetes medication is being tested in a clinical trial to enhance blood glucose control for type 2 diabetes patients.",
    "issues": []
  }
}

--- Reviewer (119460 ms) ---
{
  "thought": "Identified key topics: clinical trial, diabetes medication, type 2 diabetes. Ensured tags are specific and summary is concise.",
  "message": "A new diabetes medication is being tested in a clinical trial to enhance blood glucose control for type 2 diabetes patients.",
  "data": {
    "tags": [
      "Clinical Trials",
      "Diabetes Medication",
      "Type 2 Diabetes"
    ],
    "summary": "A clinical trial tests a new diabetes medication to improve blood glucose control in type 2 diabetes patients.",
    "issues": []
  }
}

 Finalized Output
{
  "thought": "Identified key topics: diabetes treatment, clinical trial, and blood glucose control. Formulated concise summary.",
  "message": "Tags and summary finalized based on the clinical trial details for the new diabetes medication.",
  "data": {
    "tags": [
      "Diabetes Treatment",
      "Clinical Trial",
      "Blood Glucose Control"
    ],
    "summary": "A new diabetes medication is being tested in a clinical trial to improve blood glucose control for type 2 diabetes patients.",
    "issues": []
  }
}

 Publish Package
{
  "title": "New Diabetes Drug Trial",
```

Figure 16. Final Publish Package produced by the multi-agent workflow



Short Answers Questions:

1. What were the three final tags?

Final three tags were Diabetes Treatment, Clinical Trial, and Blood Glucose Control.

2. What was the final summary?

Final summary was: “A new diabetes medication is being tested in a clinical trial to improve blood glucose control for type 2 diabetes patients.”

3. Did the Reviewer agent change anything?

Yes. The Reviewer changed both the tags & the summary from the Planner output. The Planner gave tags like- Clinical Trial Diabetes Drug & Type 2 Diabetes Treatment while the Reviewer changed them to Clinical Trials, Diabetes Medication, and Type 2 Diabetes. So the Reviewer did make some changes before the result went to the Finalizer.

Part 3: Non-determinism Experiment

In this part, I used the same clinical trial input for every run and tested the agent workflow at two temperatures: 0.7 and 0.0. I ran the workflow 20 times for each temperature, giving a total of 40 runs. I kept the model and input unchanged so that I could compare how much the final tags changed when only the temperature was different.

Fixed input used for all runs:

Title: New Diabetes Drug Trial

Content: A clinical trial is evaluating a new diabetes medication designed to improve blood glucose control in adults with type 2 diabetes.

Model: qwen3:8b

Runs: 20 at temperature 0.7 and 20 at temperature 0.0

Metric	**Temperature 0.7**	**Temperature 0.0**
Number of runs	20	20
Distinct tag sets	17	2
Tags in all 20 runs	None	Blood Glucose Control, Clinical Trials, Type 2 Diabetes
Tags in exactly 1 run	Clinical Drug Trials, Clinical Trial Drug Development, Clinical Trial Innovation, Clinical Trial Medication, Clinical Trial Research, Clinical Trial for Diabetes Medication, Diabetes Medication Development, Diabetes Medication Trial	None
P50 latency	239054.34 ms	233503.43 ms
P95 latency	315536.01 ms	255039.31 ms
P99 latency	342524.06 ms	255811.00 ms

The results showed a clear difference between the two temperatures. At temperature 0.7, I got 17 different tag sets across 20 runs, which shows that the model produced much more varied outputs. At temperature 0.0, there were only 2 different tag sets, so the results were much more consistent. The latency was also more variable at temperature 0.7, while temperature 0.0 had a lower P95 latency. Based on these runs, lowering the temperature

made the workflow more deterministic, although it did not make every run completely identical.

This kind of variation can be acceptable for tasks like generating tag suggestions, where more than one answer can still be useful. However, for a high-stakes task such as checking clinical trial safety or eligibility rules, I would want the output to be much more consistent.

During the experiment, my system ran into a disk space/resource issue and the run was interrupted before all 40 runs were finished. Since the completed raw outputs had already been saved, I updated the script so that it could skip the completed runs and continue with only the missing ones. After freeing some disk space & restarting the environment I completed all 20 runs for both temperatures.

Figure 17. Resumed non-determinism experiment after interruption using already saved outputs

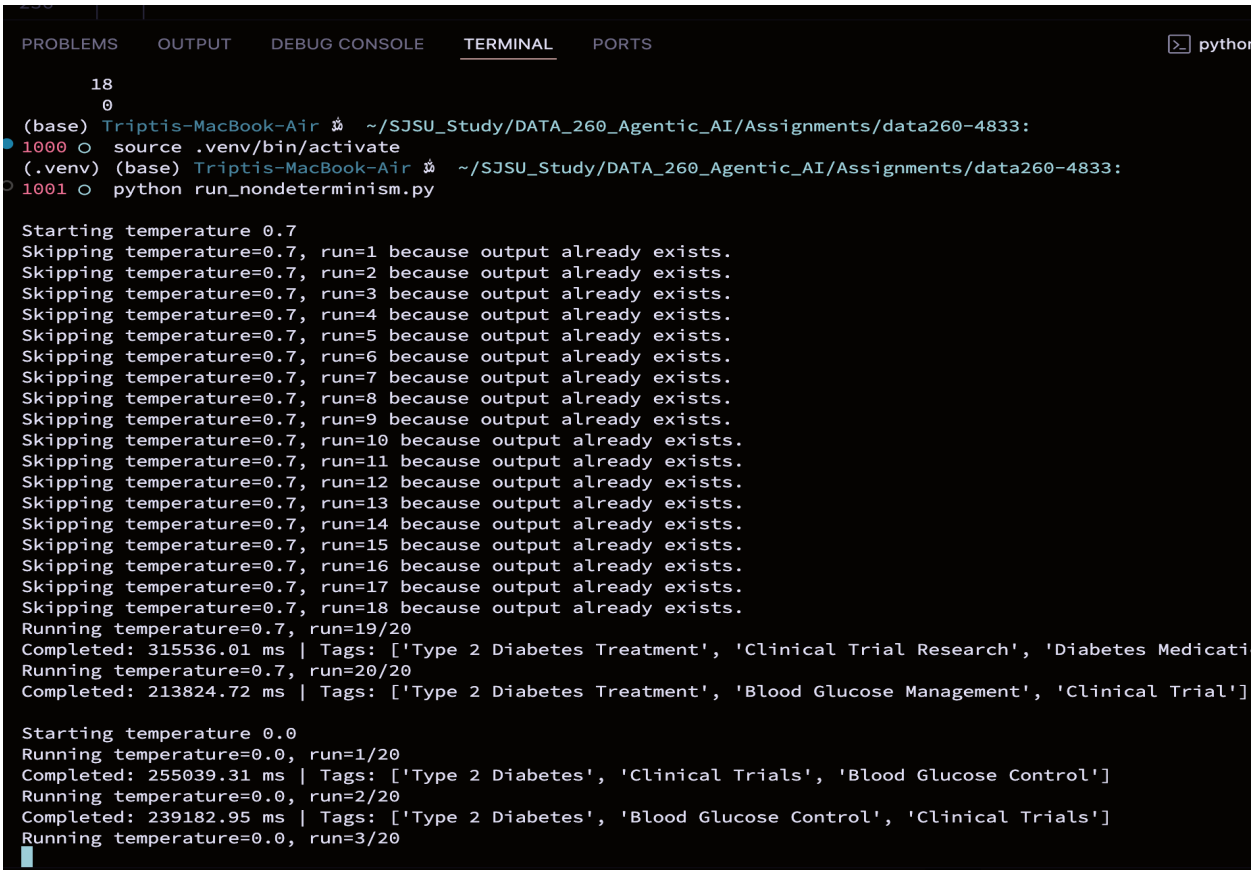
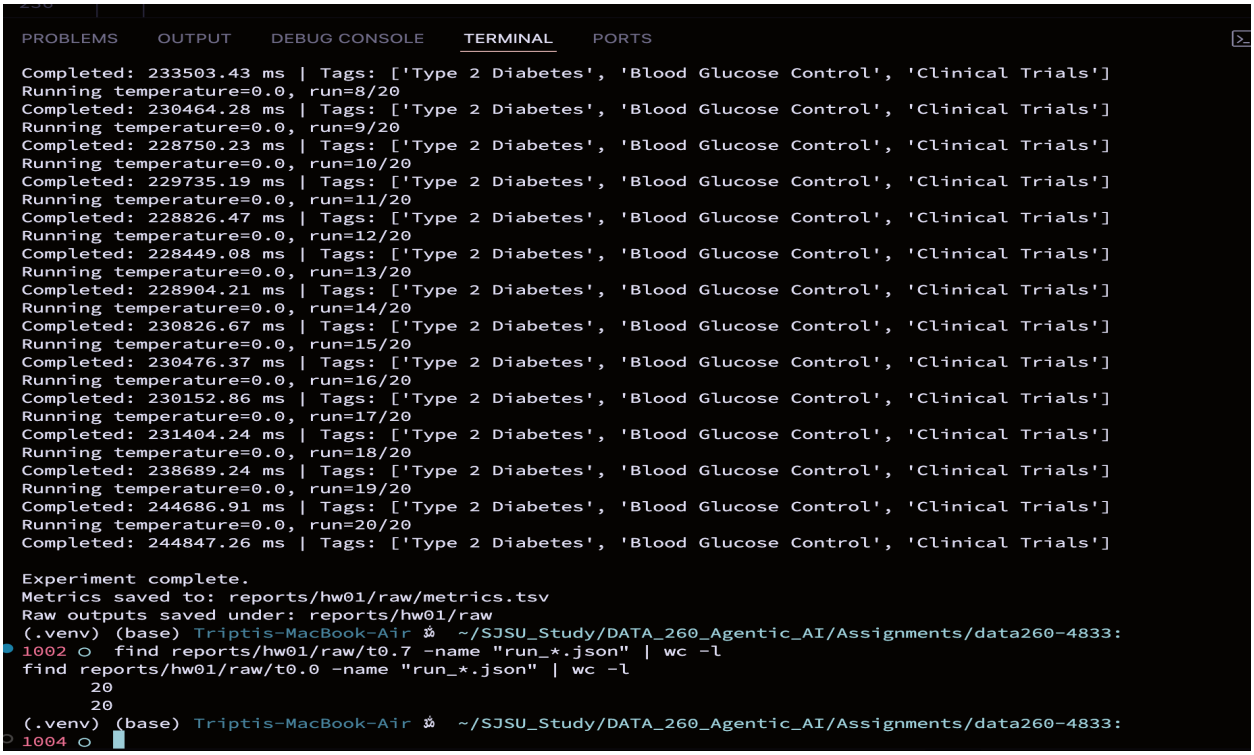


Figure 18. Completed experiment with 20 runs collected for both temperature 0.7 and temperature 0.0

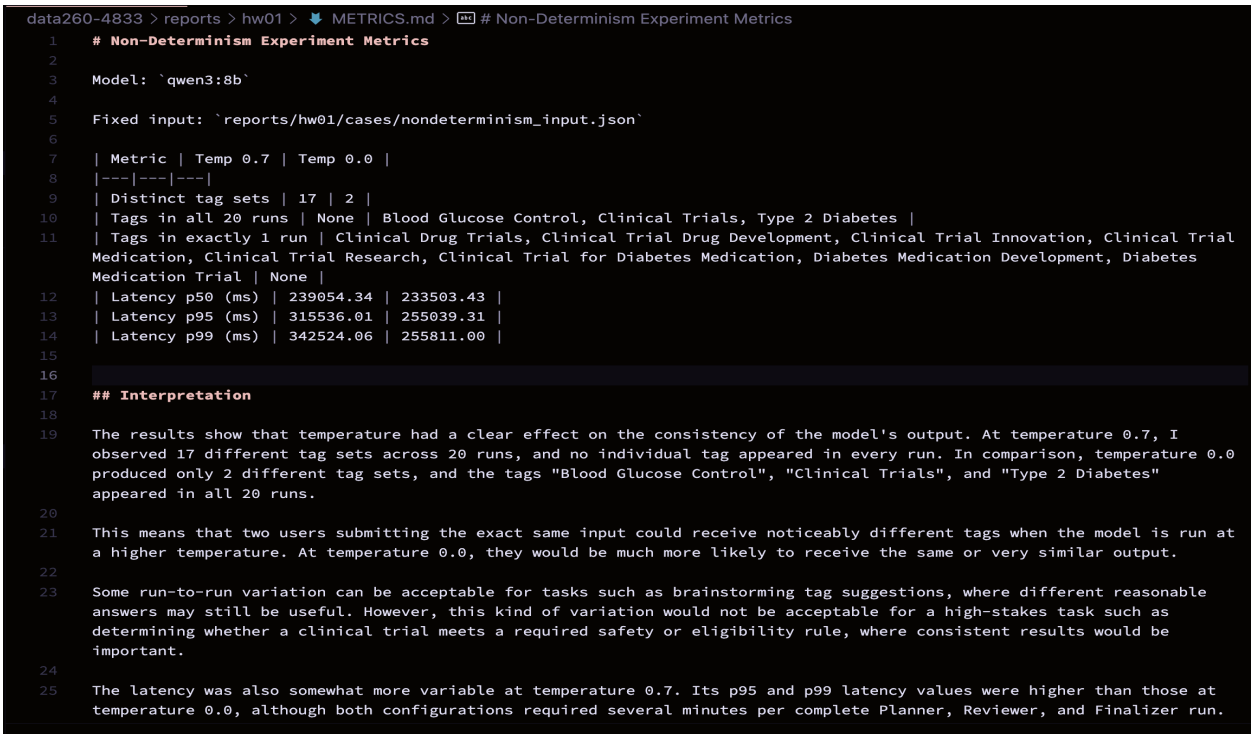


```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Completed: 233503.43 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=8/20
Completed: 230464.28 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=9/20
Completed: 228750.23 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=10/20
Completed: 229735.19 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=11/20
Completed: 228826.47 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=12/20
Completed: 228449.08 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=13/20
Completed: 228904.21 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=14/20
Completed: 230826.67 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=15/20
Completed: 230476.37 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=16/20
Completed: 230152.86 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=17/20
Completed: 231404.24 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=18/20
Completed: 238689.24 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=19/20
Completed: 244686.91 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']
Running temperature=0.0, run=20/20
Completed: 244847.26 ms | Tags: ['Type 2 Diabetes', 'Blood Glucose Control', 'Clinical Trials']

Experiment complete.
Metrics saved to: reports/hw01/raw/metrics.tsv
Raw outputs saved under: reports/hw01/raw
(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:
1002  find reports/hw01/raw/t0.7 -name "run_*.json" | wc -l
find reports/hw01/raw/t0.0 -name "run_*.json" | wc -l
      20
      20
(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:
1004
```

Figure 19. Final non-determinism metrics generated from the 40 experiment runs



data260-4833 > reports > hw01 > METRICS.md > # Non-Determinism Experiment Metrics

```
# Non-Determinism Experiment Metrics

Model: `qwen3:8b`

Fixed input: `reports/hw01/cases/nondeterminism_input.json`

| Metric | Temp 0.7 | Temp 0.0 |
|---|---|---|
| Distinct tag sets | 17 | 2 |
| Tags in all 20 runs | None | Blood Glucose Control, Clinical Trials, Type 2 Diabetes |
| Tags in exactly 1 run | Clinical Drug Trials, Clinical Trial Drug Development, Clinical Trial Innovation, Clinical Trial Medication, Clinical Trial Research, Clinical Trial for Diabetes Medication, Diabetes Medication Development, Diabetes Medication Trial | None |
| Latency p50 (ms) | 239054.34 | 233503.43 |
| Latency p95 (ms) | 315536.01 | 255039.31 |
| Latency p99 (ms) | 342524.06 | 255811.00 |

## Interpretation

The results show that temperature had a clear effect on the consistency of the model's output. At temperature 0.7, I observed 17 different tag sets across 20 runs, and no individual tag appeared in every run. In comparison, temperature 0.0 produced only 2 different tag sets, and the tags "Blood Glucose Control", "Clinical Trials", and "Type 2 Diabetes" appeared in all 20 runs.

This means that two users submitting the exact same input could receive noticeably different tags when the model is run at a higher temperature. At temperature 0.0, they would be much more likely to receive the same or very similar output.

Some run-to-run variation can be acceptable for tasks such as brainstorming tag suggestions, where different reasonable answers may still be useful. However, this kind of variation would not be acceptable for a high-stakes task such as determining whether a clinical trial meets a required safety or eligibility rule, where consistent results would be important.

The latency was also somewhat more variable at temperature 0.7. Its p95 and p99 latency values were higher than those at temperature 0.0, although both configurations required several minutes per complete Planner, Reviewer, and Finalizer run.
```

Part 4: Model Client and Token Accounting

In this part, I created a reusable model client in `src/model_client.py` and used it from `hw1_client.py`. The system instructions were loaded from `AGENT.md`, which asked the model to respond using only bullet points for code review requests. I then ran a five-turn conversation & checked `/stats` after turn 3 and again after turn 5.

The purpose of this part was to understand how conversation history affects token usage. So instead of sending only the newest message, the previous system, user & assistant messages were included again so that the model could keep the context of the conversation.

Figure 20. Reusable model client implemented in src/model_client.py

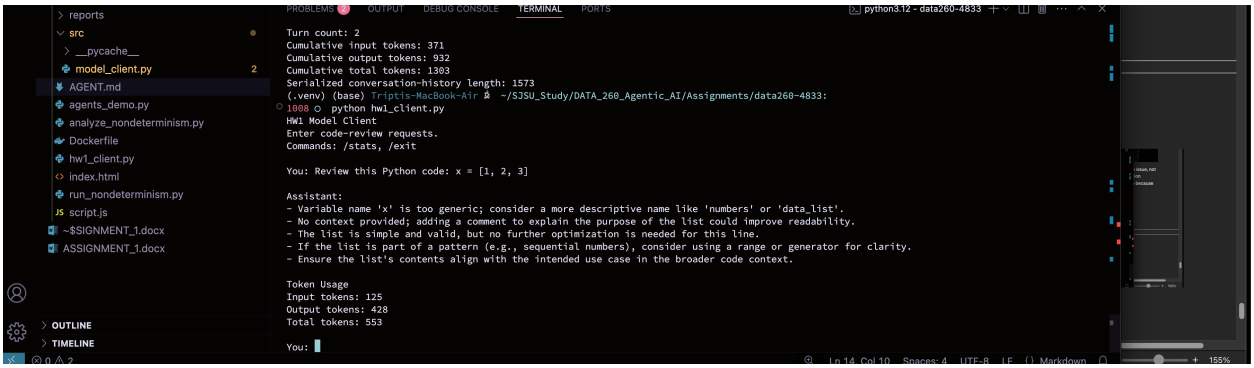
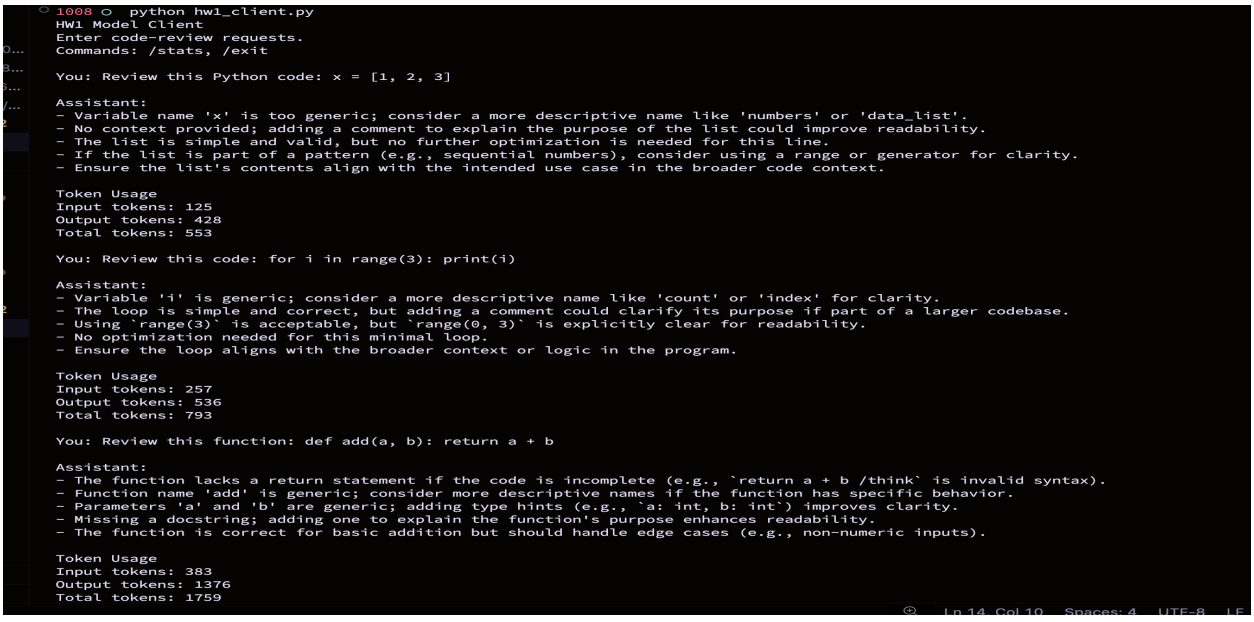


Figure 21. Five-turn conversation following the bullet-only instructions from AGENT.md



```
You: /stats

Conversation Stats
Turn count: 3
Cumulative input tokens: 765
Cumulative output tokens: 2340
Cumulative total tokens: 3105
Serialized conversation-history length: 2304

You:
```

Ln 14, Col 10 Spaces: 4 UTF-8 LF

```
You: Review this code: numbers = [x * x for x in range(5)]

Assistant:
- Variable 'numbers' could be more descriptive (e.g., 'squares') to clarify the list's contents.
- List comprehension is concise but could use a comment if part of a larger codebase for clarity.
- `range(5)` generates 0-4, which is correct, but ensure it aligns with the intended output range.
- No optimization needed for this simple list comprehension.
- Consider using a generator expression if memory efficiency is a priority for large ranges.

Token Usage
Input tokens: 532
Output tokens: 372
Total tokens: 904

You: Review this function: def greet(name): print("Hello " + name)

Assistant:
- The function lacks a return statement; consider returning the greeting string instead of printing for reusability.
- No docstring provided; adding one to describe the function's purpose improves readability.
- Parameter 'name' is clear, but adding a type hint (e.g., `name: str`) enhances type clarity.
- The function is correct as-is but could be more flexible if the caller needs to handle the output (e.g., logging, UI display).
- Ensure the function aligns with the broader codebase's design (e.g., whether printing or returning is preferred).

Token Usage
Input tokens: 658
Output tokens: 554
Total tokens: 1212

You: /stats

Conversation Stats
Turn count: 5
Cumulative input tokens: 1955
Cumulative output tokens: 3266
Cumulative total tokens: 5221
Serialized conversation-history length: 3582
```

Figure 22. Token statistics displayed during the conversation using /stats

```
You: /exit

Final Conversation Stats

Conversation Stats
Turn count: 5
Cumulative input tokens: 1955
Cumulative output tokens: 3266
Cumulative total tokens: 5221
Serialized conversation-history length: 3582
(.venv) (base) Triptis-MacBook-Air  ~/SJSU_Study/DATA_260_Agentic_AI/Assignments/data260-4833:
1008
```

Short Answers:

1. Why is prior conversation context resent with every turn?

The model does not automatically remember previous messages between separate requests. To continue the same conversation the earlier messages have to be sent again along with the new message so that the model knows what was discussed before.

2. How is a system prompt different from a user message?

A system prompt gives the model instructions about how it should behave during the conversation. A user message contains the actual question or request. In my experiment, AGENT.md was used as the system prompt and instructed the model to give bullet-only code reviews.

3. Why do input tokens grow over a conversation?

Input tokens increase because each new request includes the previous conversation history along with the latest message. As the conversation becomes longer, more text has to be sent back to the model.

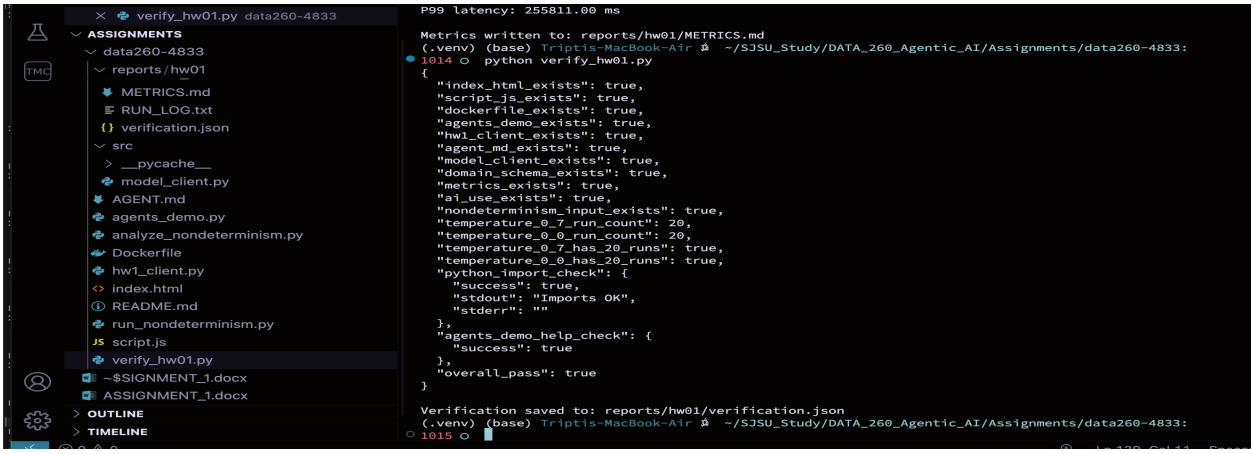
4. What eventually limits that growth?

Model has a limited context window so there is a max amount of text that can be included in one request. Once the conversation becomes too large, older content has to be removed, summarized or reduced to stay within that limit.

VERIFICATION:

After completing the implementation, I ran the provided verification script to check the required files and functionality. The verification output was saved to reports/hw01/verification.json. The final result showed that the required checks passed successfully.

Figure 23. Homework verification completed successfully with `overall_pass: true`



AI Use Declaration:

1. What did I use an AI assistant for, and what did I do myself?

I used an AI assistant mainly for guidance while working through the assignment, especially when I was learning unfamiliar concepts related to LangChain, Ollama & the model client.

I created & tested HTML form, JavaScript validation, Docker application, local Ollama setup, agent pipeline, non-determinism experiment & model-client conversation on my own machine. I also ran all of the experiments myself & used the actual outputs from my runs for the metrics reported in this assignment.

2. One AI-produced output that was wrong or unsuitable

During the model-client experiment, the local Qwen3 model incorrectly mentioned a `/think` suffix while reviewing Python code even though I had not included `/think` in the code I submitted.

3. How did I detect or verify the problem?

I compared the model's review with the exact Python code I had entered in the terminal. Since my input did not contain `/think`, I could confirm that this part of the model response was incorrect and was not caused by my Python code.

I also verified my Python environment separately by checking the active interpreter and testing the LangChain imports directly from the terminal.

4. What did I change, and why does it work now?

I added `/no_think` to the instructions in `AGENT.md` & restarted the conversation so the previous incorrect response would not remain in the new conversation history.

After this change the model followed the required bullet-only review format more consistently. I also configured VS Code to use the same `.venv` Python interpreter that successfully ran the program from the terminal.

GitHub Repository and Final Submission:

I created a private GitHub repository named `data260-4833` and pushed the completed assignment files to the main branch. The repository contains the application files, agent code, model client, experiment outputs, metrics, verification results, and supporting documentation.

Figure 24. Final Homework 1 GitHub repository with the hw1 submission tag